Evaluation and Results

1. Evaluation Overview

To ensure the reliability, accuracy, and factual integrity of the AI Sarthi Healthcare Assistant, a robust evaluation framework was implemented. The system utilizes both automated metrics to measure lexical and semantic overlap with reference data, and manual evaluation frameworks to assess subjective qualities like faithfulness and completeness.

2. Experimental Setup

- **Base Model:** microsoft/Phi-3-mini-4k-instruct
- **Fine-Tuned Adapter:** Custom QLoRA adapter
- **Embedding Model:** BAAI/bge-small-en-v1.5
- **Vector Database:** ChromaDB
- **Evaluation Scripts:** scripts/evaluate_model.py, scripts/evaluation/compare_models.py
- **Hardware:** Local CUDA-enabled GPU (specific GPU architecture varies based on the deployment environment).

3. Evaluation Dataset

A synthetic instruction dataset was derived from raw healthcare documents to benchmark the model.

- **Total Valid Samples:** 3,979 QA pairs.
- **Test Split:** 75 high-quality samples reserved for evaluation (data/prepared/instruction_test.jsonl).
- **Data Categories:** The dataset spans 11 unique categories, predominantly covering 'General Healthcare' and 'Statistics' from documents like the *Health Ministry Annual Report* and *NFHS-5 National Report*.

4. Baselines

The primary comparison baseline is the generic, un-fine-tuned microsoft/Phi-3-mini-4k-instruct model interacting with the identical RAG pipeline. Additionally, scripts explicitly support comparing results against other architectures (e.g., gemma3).

5. Evaluation Metrics

The framework (src/evaluation/metrics.py) defines the following metrics:

- **BLEU (Bilingual Evaluation Understudy):** Measures exact lexical overlap between the generated text and the reference text using n-gram precision. Used to determine if exact phraseology is preserved.
- **ROUGE-L (Recall-Oriented Understudy for Gisting Evaluation - Longest Common Subsequence):** Measures recall and structural similarity. Used to assess if the generated answer captures the core facts of the reference.
- **BERTScore F1:** Measures semantic similarity by leveraging contextual embeddings from pre-trained language models. Used to ensure the meaning of the answer aligns with the reference, even if different vocabulary is utilized.
- **Latency (seconds):** Tracked directly in evaluate_model.py to ensure the system is responsive enough for real-time chat.

6. Automated Evaluation

The scripts/evaluate_model.py runner iterates over the test set, capturing latency and answer generation.
Note: Specific tabular numerical results (exact BLEU, ROUGE, BERTScore values) for the final run are not explicitly documented in the current repository output logs.

7. Manual Evaluation

A manual evaluation framework is facilitated by scripts/evaluation/create_manual_evaluation_template.py, generating templates for human reviewers.

- **Evaluation Criteria:**
  - **Faithfulness (1-5):** Does the model's answer strictly adhere to the retrieved context without hallucinating external information?
  - **Completeness (1-5):** Does the model provide a comprehensive answer to the user's query based on the available data?
- *Results data for the manual evaluation phase is not explicitly present in the repository.*

8. Model Comparison

The compare_models.py script aggregates evaluation outputs (e.g., results_phi3.csv vs results_gemma3.csv) to compute:

- Average, Median, Min, and Max Latency.
- Average Source Count.
- Category-wise metric breakdowns.

9. Graphs

The system automatically generates evaluation and dataset visualizations stored in the plots/ directory:

- **Training Artifacts (plots/training_plots/):**
  - loss.jpeg: Training loss over epochs.
  - learning_rate.jpeg: Learning rate schedule.
  - grad_norm.jpeg: Gradient normalization trends.
- **Dataset Artifacts (plots/):**
  - quality_distribution.png: Demonstrates the distribution of instruction quality (Average: 4.38/5).
  - category_distribution.png: Shows the breakdown of topics covered in the test set.

10. Results Discussion

Based on the training artifacts and dataset analysis reports:

- The QLoRA training successfully converged, as evidenced by the consistent decrease shown in loss.jpeg.
- The RAG pipeline successfully extracts sources, mapping dynamically generated chunks back to their origin files (e.g., Health_Ministry_Annual_Report_25-26.pdf), directly addressing the requirement for citation-backed responses.
- The average length of reference outputs in the training dataset is 74.34 words, conditioning the model to produce concise, highly focused answers suitable for a chatbot interface.

11. Error Analysis

Based on the known limitations of similar RAG architectures handled in the pipeline:

- **Missing Context:** If ChromaDB fails to retrieve the top-k chunks containing the relevant fact, the model is instructed via the prompt builder to state it cannot answer, preventing hallucination.
- **Context Truncation:** Large retrieved chunks can exceed the model's context window; therefore, LangChain text splitters enforce strict chunk boundaries during ingestion.

12. Threats to Validity

- **Automated Metric Limitations:** Lexical metrics like BLEU often penalize correct answers that use synonyms. Semantic metrics (BERTScore) are preferred but require more computational overhead to calculate.
- **Dataset Size:** While 3,979 samples are robust for adapter fine-tuning, the 75-sample test set provides a relatively small benchmark for comprehensive generalization metrics.

13. Final Findings

The AI Sarthi system provides a structurally sound, highly modular architecture for healthcare question answering. The combination of PyTorch QLoRA fine-tuning and a ChromaDB semantic retrieval pipeline allows the model to produce responses grounded directly in verifiable healthcare policies, fulfilling the primary project objectives.